

The Only Docker Tutorial You Need To Get Started

Here are your quick-reference notes from *The Coding Sloth's* Docker tutorial to help you easily recall the concepts and commands in the future.

1. Core Concepts: The "Why" and "What"

- **The Problem:** *"It works on my machine!"* Variations in environments, operating systems, and local dependencies cause code to fail when shared.
- **The Solution (Docker):** Packages code, runtime, dependencies, and configurations into a lightweight container that runs identically on any computer [00:30].
- **The Two Key Pillars:**
 - **Image (The Recipe):** A read-only template containing instructions, runtime tools, and code required to build your app [01:41].
 - **Container (The Meal):** A live, isolated instance running the image recipe. You can spin up multiple containers from a single image [01:52].

2. Basic Setup & Testing

- **Install:** Download **Docker Desktop** (provides a GUI for easy management and includes the Command Line Interface/CLI) [02:58].
- **Verify Installation:** Run `docker --version` in your terminal [03:12].
- **IDE Tip:** Install the **Docker extension** in your IDE (like VS Code) for syntax support and metadata documentation hints [03:22].

3. Writing a Dockerfile (The Recipe)

Using a Node.js server as the reference, here is how a step-by-step Dockerfile is built [03:32]:

```
# 1. Base Image - Setting the foundational environment
FROM node:22

# 2. Working Directory - Telling Docker where our app lives inside the image
WORKDIR /app

# 3. Optimize Layer Caching - Copy package files and install dependencies FIRST
# Docker caches layers; if code changes but dependencies don't, it skips npm install!
COPY package*.json ./
RUN npm install

# 4. Copy Application Code - Bring the rest of your local files over
COPY . .

# 5. Environment Variables & Ports
ENV PORT=3000
EXPOSE 3000

# 6. Start the Container - CMD is executed when running, RUN is for building
CMD ["npm", "start"]
```

Crucial Optimization: .dockerignore

Always create a .dockerignore file next to your Dockerfile to ignore massive directories like node_modules. This prevents dragging heavy local build files into the clean image build [05:33].

4. Image & Container Commands

Once the recipe is written, use these CLI commands to build and run [06:24]:

- **Build the Image:**

Bash

```
docker build -t my-app-name .
```

- -t: Tags/names your image.
- .: Specifies the build context (current folder) [06:34].

- **Run the Container:**

Bash

```
docker run -p 3000:3000 my-app-name
```

- -p 3000:3000: **Port Forwarding**. Maps [Host Computer Port] : [Container Internal Port]. The container won't accept local traffic without this bridge [06:55].

5. Advanced: Multi-Container Apps, Compose, & Volumes

For real-world web apps that require both a server and a database (e.g., PostgreSQL), putting everything in one container is bad practice. Instead, use multiple containers and orchestrate them [08:23].

Docker Compose (compose.yaml)

Automates spinning up, linking, and configuring multiple container ecosystems at once [09:03]:

YAML

services:

backend:

build: .

ports:

- "3000:3000"

database:

image: postgres

environment:

POSTGRES_USER: user

POSTGRES_PASSWORD: password

volumes:

- postgres_data:/var/lib/postgresql/data

```
volumes:  
  postgres_data:
```

What are Docker Volumes?

Containers are *ephemeral* (state/data resets whenever a container restarts). **Volumes** map a persistent folder on your local machine to the container so database state is retained across runs [10:07].

- **Start the app cluster:** docker compose up [10:40]
- **Stop the app cluster:** docker compose down [10:47]

6. Built-in Tools to Remember

- **Debugging:** Use Docker Desktop GUI or the docker exec command to open up an active container's console, look at live execution logs, and monitor performance [07:21].
- **Docker Scout:** Scans your built images to find security vulnerabilities in package dependencies and suggests remediation paths [07:44].
- **Docker Build Cloud:** Offloads image compiling to cloud servers, reducing build times by up to 39x and allowing teams to share a mutual build cache [10:54].
- **The Ultimate Cheat Code:** If you don't want to type your Docker configuration files from scratch, simply run docker init in your terminal, and Docker will scaffold your baseline Dockerfile, .dockerignore, and compose.yaml files automatically [11:45].